Name: Anubhav Kumar Thakur
Roll no: 2301010002
Sec: A
Cloud computing

Handwritten project Reflection

Project Reflection

1. What i implemented

In this project, I developed and deployed a light-weight command parsing language model using Knowledge distillation. I used phi-3.5-mini-instruct as the teacher model to generate structured command-parsing responses, and these teacher-student generated examples were then used to train an EleutherAI/pythia-70M student model.

I trained the student model using 1500 teacher-generated examples and evaluated it on the provided validation dataset. The final model achieved 99% JSON validity, successfully meeting the required 95% quality gate.

After training the model, I deployed it as a FastAPI service using docker and kubernetes. I used seaweedFS as S3-compatible object storage so that the model could be stored separately from the application image and downloaded when the application started.

This helped me understand how a machine-learning model can be treated as a versioned deployment artifact rather than being directly embedded into the application.

2. What I learned

This project helped me understand the complete lifecycle of a machine-learning application, starting from dataset generation and model training and continuing through deployment, testing, monitoring and recovery.

I learned how different Kubernetes components work together, In particular, I gained practical experience with Deployment, Services, Ingress, readiness probes, liveness probes and Horizontal Pod Autoscaling (HPA).

I also learned that deployment a machine-learning model is significantly different from simply running the model locally. In a real deployment, factors such as a model storage, startup time, memory and CPU requirements, health checks, request routing and failure recovery all become important.

The CI/CD implementation was another important learning experience. I learned how model evaluation can be incorporated into an automated pipeline rather than treating model quality as a separate model setup.

3.

Problems I faced.

One of the major problems i encountered during training was undesirable repetition and EOS behaviour in an early configuration. The model sometimes produced repeated or prematurely terminated outputs. I addressed this by using the complete sequence for the causal language model objective, retraining from a fresh model and using a more conservative learning rate.

I also faced several practical deployment challenges. I had to configure SeaweedFS, synchronize the trained model files to S3-combatible storage, configure Kubernetes secrets and make sure that the model could download and load the model correctly during startup.

4.

Future improvements.

I will continue to working on this project, first priority would be reducing inference latency. Techniques such as model quantization and inference optimization could reduce CPU and memory requirements. and potentially improve response time.

I would also work on improving the model's semantic accuracy, particularly in full-frame accuracy.